PyTupli: Enabling Collaboration in Offline Reinforcement Learning

10 December 2025

Summary

Offline reinforcement learning (RL) offers a powerful way to derive effective decision-making policies for control problems using pre-collected data. However, managing and sharing such datasets in collaborative research settings can be challenging, as proper versioning, filtering, and access control are required. PyTupli is a free, Python-based toolkit designed to support this workflow by providing an easy-to-use yet specialized framework that remains independent of external service providers. Unlike centralized platforms, PyTupli is designed for self-hosted deployment, giving research teams full control over their data infrastructure and access policies. Its client library allows users to serialize and store control problems, upload new data, and retrieve precisely the subsets they need through flexible and expressive filters. Built-in metrics help evaluate dataset coverage and utility, informing both dataset selection and algorithm design for offline RL. To ensure secure deployment, a container-based server component offers authentication, role-based access control, and automated certificate provisioning. Together, these capabilities enable researchers to create, manage, exchange, and analyze datasets for offline RL in a robust and accessible manner.

Statement of need

Reinforcement learning (RL) provides powerful methods for decision-making under uncertainty, but training RL agents typically requires extensive interaction with the underlying system or a computationally expensive simulator. Offline RL alleviates this requirement by training agents solely on previously collected data (Lange, Gabel, and Riedmiller 2012). Such datasets contain tuples of state, action, next state, and reward, obtained from real systems or simulators, which we refer to as tuple datasets.

Effectively managing, sharing, and curating these datasets is essential for collaborative offline RL research, yet existing tools provide only partial support.

Platforms like Zenodo or the free version of HuggingFace allow users to share finalized datasets but are not suitable for ongoing or private collaborations. Furthermore, they lack mechanisms for tracking dataset structure or performing efficient queries. Traditional databases are better suited but require expertise to design and maintain workflows.

PyTupli addresses this gap by providing a Python toolkit for creating, storing, and sharing tuple datasets for custom environments. We provide a Docker container that can be deployed independently to maintain control over data. Each dataset is associated with a benchmark, stored as JSON and linked to artifacts such as time-series data, hyperparameters, or trained policies. PyTupli includes integrated user and access management.

Since offline RL performance depends on dataset quality (Schweighofer et al. 2022; Suttle, Suresh, and Nieto-Granda 2025; Asadulaev, Karray, and Takac 2025), PyTupli offers extensive filtering capabilities (Younis et al. 2024; Liu et al. 2023). PyTupli enables tuple-level filtering, allowing rebalancing datasets or selecting transitions from specific regions of the state space. In addition, PyTupli provides metrics for assessing dataset coverage and reward characteristics, which can predict performance (Schweighofer et al. 2022; Asadulaev, Karray, and Takac 2025) and guide algorithm selection.

State of the Field

Publicly available tuple datasets have been essential for advancing offline RL algorithms (Kumar et al. 2020; Kostrikov, Nair, and Levine 2021). These collections span domains such as robotics, games (Fu et al. 2020; Younis et al. 2024; Formanek et al. 2023; Gulcehre et al. 2020), power systems (Qin et al. 2022), and autonomous driving (Liu et al. 2023; Lee, Eom, and Kwon 2024). As offline RL matures, it is applied to more diverse, task-specific problems. However, no toolbox supports collaborative dataset creation and sharing for custom environments.

Minari (Younis et al. 2024) is the closest related tool, offering standardized datasets and functionality for filtering and recording interactions. However, it focuses on distributing datasets for established benchmarks. Although Minari permits users to request publication of custom datasets on its official platform, this approach is often unsuitable for ongoing or proprietary work or for datasets that evolve over time. PyTupli instead targets collaborative workflows for custom control tasks, enabling teams to share datasets without relying on a central server. Additionally, it provides tools for assessing dataset quality or coverage.

Core Functionalities

We briefly describe the core functionalities of PyTupli which are illustrated in Figure 1.

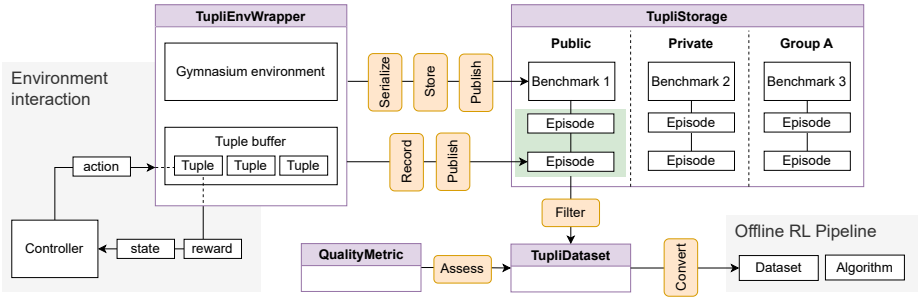

Figure 1: Overview of the core functionalities of PyTupli.

Benchmark and Artifact Management: PyTupli enables users to store any control task defined as a gymnasium environment. Environments may include parameterizable configurations that produce variations in task dynamics. A fully specified environment is stored as a benchmark, providing a unique reference for reproducible evaluation of controllers. Benchmarks can reference additional data, such as exogenous inputs, time-series data, or pre-trained models. These external units, referred to as artifacts, are stored independently and linked to multiple benchmarks to avoid duplication.

Data Management: PyTupli supports ingesting, storing, and querying structured datasets (RL tuples), including their relation to existing benchmark problems and any relevant metadata. MongoDB serves as the backend, providing scalable storage and retrieval. Table 1 shows how ingestion and retrieval times scale with dataset size. Since benchmarking was performed against a locally deployed server, network latency is excluded from all reported times and will add to these figures in practice.

Multi-User Collaboration and Access Control: PyTupli facilitates collaborative workflows through private, group, and public scopes. Based on their assigned role, users can store, retrieve, delete, and publish objects. A server-side backend with FastAPI provides a REST interface for secure, programmatic access, while token-based authentication ensures secure sharing across teams or organizations.

Integration with Existing Offline RL Infrastructure: An interface to the gymnasium framework enables users to record interactions with gymnasium environments as RL tuples. Furthermore, retrieved tuple datasets are made available in a form that can easily be converted into the dataset formats used by existing offline RL libraries such as d3rlpy (Seno and Imai 2022).

Assessment of Dataset Quality: PyTupli exposes dataset quality metrics through its client API, allowing users to assess dataset suitability without leaving the workflow. These metrics are implemented from the offline RL literature and require no additional setup beyond a retrieved dataset.

Table 1: Upload and download times for established datasets averaged over 10 runs, measured on a consumer laptop with a local server deployment. We chose two examples with low, medium, and high dataset size from the Minari collection, where M denotes the number of episodes and N the total number of tuples. However, not only the size, but also the nature of observations has a strong influence on processing times.

Dataset	Size	M	N	Upload (s)	Download (s)
D4RL					
door/human-v2	3.5MB	25	7K	0.83	0.24
hammer/human-v2	6.2MB	25	11K	1.11	0.40
antmaze/medium-play-v1	605.2MB	1K	1M	173.26	126.62
Atari					
pitfall/expert-v0	351.7MB	10	65K	18.56	16.51
Mujoco					
ant/expert-v0	1.92GB	2K	2M	64.17	29.65
humanoid/expert-v0	2.95GB	1K	999K	96.61	55.32

Quality Metrics

Return-Based Metrics

PyTupli computes trajectory quality (TQ) (Schweighofer et al. 2022) and average Q-value (Asadulaev, Karray, and Takac 2025), which inform algorithm choice for a given dataset. High TQ favors behavioral cloning, while low TQ favors value-based methods. Estimated return improvement (Swazinna, Udluft, and Runkler 2021) relates maximum and average returns. Average Q-value operates at the tuple level and is a strong predictor of performance, requiring a Q-function fitted via Bellman updates.

Coverage-Based Metrics

PyTupli also computes state-action coverage metrics, since low coverage is known to reduce offline RL performance (Schweighofer et al. 2022). Supported metrics include an entropy approximation via unique state-action pairs (Schweighofer et al. 2022) and behavioral entropy (Suttle, Suresh, and Nieto-Granda 2025), which uses density-based weighting of state-action space regions.

Software Design

PyTupli is designed around a clear separation between a lightweight client library and a centralized server backend. On the client side, PyTupli integrates

via a gymnasium environment wrapper, which is a well-established abstraction in the RL ecosystem. The server component exposes a REST API that implements functionality specific to offline RL datasets, such as structured storage of benchmarks, episodes, and artifacts, as well as flexible filtering and access control. A centralized service was favored over object storage or git-based workflows because offline RL datasets require domain-specific querying and metadata handling that these approaches do not natively support. Deployment is simplified via a Docker Compose setup, providing a production-ready stack that can be launched with minimal configuration. Here, the API server is hidden behind an Nginx web server acting as a reverse proxy for increased scalability. We chose MongoDB as the backend due to its ability to store heterogeneous, evolving data without rigid schemas while supporting efficient indexing for nested fields. GridFS enables integrated storage of large artifacts, avoiding external dependencies.

Existing tools such as Minari focus on distributing pre-existing datasets for offline RL. PyTupli instead targets research groups that need to create, host, and curate custom benchmarks collaboratively. This fundamental difference in scope and architecture made contributing to existing projects impractical, motivating the development of new software tailored to this use case.

Research Impact Statement

Since its release on PyPI in May 2025, PyTupli has been downloaded approximately 65 times per month, suggesting early uptake. The project is actively maintained by two contributors with automated testing and CI/CD pipelines. Documentation is available via ReadTheDocs, and tutorials provide practical guidance. PyTupli has been integrated into the CommonPower framework, demonstrating applicability in real research settings.

AI Usage Disclosure

Visual Studio Code with Claude Sonnet 4.5 was used for scaffolding tests. GPT 5.2 was used for refining the manuscript.

Acknowledgements

This work was partially supported by the German Research Foundation (AL 1185/9-1).

References

Asadulaev, Arip, Fakhri Karray, and Martin Takac. 2025. “Expert or Not? Assessing Data Quality in Offline Reinforcement Learning.” *arXiv Preprint*

arXiv:2510.12638. https://doi.org/10.48550/arXiv.2510.12638.

Formanek, Claude, Asad Jeewa, Jonathan Shock, and Arnu Pretorius. 2023. "Off-the-Grid MARL: Datasets and Baselines for Offline Multi-Agent Reinforcement Learning," 2442–44. https://doi.org/10.65109/vmil8420.

Fu, Justin, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. 2020. "D4RL: Datasets for Deep Data-Driven Reinforcement Learning." *arXiv Preprint arXiv:2004.07219*. https://doi.org/10.48550/arXiv.2004.07219.

Gulcehre, Caglar, Ziyu Wang, Alexander Novikov, Thomas Paine, Sergio Gómez, Konrad Zolna, Rishabh Agarwal, et al. 2020. "RL Unplugged: A Suite of Benchmarks for Offline Reinforcement Learning." *Advances in Neural Information Processing Systems* 33: 7248–59. https://dl.acm.org/doi/abs/10.5555/3495724.3496332.

Kostrikov, Ilya, Ashvin Nair, and Sergey Levine. 2021. "Offline Reinforcement Learning with Implicit Q-Learning." *arXiv Preprint arXiv:2110.06169*. https://doi.org/10.48550/arXiv.2110.06169.

Kumar, Aviral, Aurick Zhou, George Tucker, and Sergey Levine. 2020. "Conservative Q-Learning for Offline Reinforcement Learning." *Advances in Neural Information Processing Systems* 33: 1179–91. https://dl.acm.org/doi/abs/10.5555/3495724.3495824.

Lange, Sascha, Thomas Gabel, and Martin Riedmiller. 2012. "Batch Reinforcement Learning." In *Reinforcement Learning: State-of-the-Art*, 45–73. Springer. https://doi.org/10.1007/978-3-642-27645-3_2.

Lee, Dongsu, Chanin Eom, and Minhae Kwon. 2024. "AD4RL: Autonomous Driving Benchmarks for Offline Reinforcement Learning with Value-Based Dataset." In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 8239–45. IEEE. https://doi.org/10.1109/ICRA57147.2024.10610308.

Liu, Zuxin, Zijian Guo, Haohong Lin, Yihang Yao, Jiacheng Zhu, Zhepeng Cen, Hanjiang Hu, et al. 2023. "Datasets and Benchmarks for Offline Safe Reinforcement Learning." *arXiv Preprint arXiv:2306.09303*. https://doi.org/10.48550/arXiv.2306.09303.

Qin, Rong-Jun, Xingyuan Zhang, Songyi Gao, Xiong-Hui Chen, Zewen Li, Weinan Zhang, and Yang Yu. 2022. "NeoRL: A Near Real-World Benchmark for Offline Reinforcement Learning." *Advances in Neural Information Processing Systems* 35: 24753–65. https://doi.org/10.52202/068431-1795.

Schweighofer, Kajetan, Marius-constantin Dinu, Andreas Radler, Markus Hofmarcher, Vihang Prakash Patil, Angela Bitto-Nemling, Hamid Eghbal-Zadeh, and Sepp Hochreiter. 2022. "A Dataset Perspective on Offline Reinforcement Learning." In *Conference on Lifelong Learning Agents*, 470–517. PMLR. https://doi.org/10.48550/arXiv.2111.04714.

Seno, Takuma, and Michita Imai. 2022. "d3rlpy: An Offline Deep Reinforcement Learning Library." *Journal of Machine Learning Research* 23 (315): 1–20. https://dl.acm.org/doi/abs/10.5555/3586589.3586904.

Suttle, Wesley A, Aamodh Suresh, and Carlos Nieto-Granda. 2025. "Behavioral Entropy-Guided Dataset Generation for Offline Reinforcement Learn-

ing." *arXiv Preprint arXiv:2502.04141*. https://doi.org/10.48550/arXiv.2502.04141.

Swazinna, Phillip, Steffen Udluft, and Thomas Runkler. 2021. "Measuring Data Quality for Dataset Selection in Offline Reinforcement Learning." In *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*, 1–8. IEEE. https://doi.org/10.1109/SSCI50451.2021.9660006.

Younis, Omar G., Rodrigo Perez-Vicente, John U. Balis, Will Dudley, Alex Davey, and Jordan K Terry. 2024. *Minari* (version 0.5.0). Zenodo. https://doi.org/10.5281/zenodo.13767625.